



Zoho Schools for Graduate Studies



Notes

Generics:

Generics allow classes and methods to operate on different data types while providing type safety.

Example:

The CustomList<T> class is a generic class that can store elements of any type. By using <T>, the same class works for String, Integer, Double, or any custom class.

```
1 package part3;
2
3 import java.util.*;
4
5 public class CustomList<T> {
6     List<T> al=new ArrayList<T>();
7
8     public void addElement(T s) {
9         al.add(s);
10    }
11
12    public void removeElement(T s) {
13        al.remove(s);
14    }
15
16    public String toString() {
17        return al.toString();
18    }
```

- Without overriding toString(), printing the object shows its memory reference.

```
2
3 public class CustomListDemo {
4     public static void main(String[] args) {
5         CustomList<String> cl=new CustomList<>() {
6             cl.addElement("one");
7             cl.addElement("two");
8             cl.addElement("three");
9             cl.removeElement("two");
10            System.out.println(cl); // [one, two, three]
11
12            CustomList<Integer> cl1=new CustomList();
13            cl1.addElement(12); // auto-boxing
14            cl1.addElement(new Integer(22)); // auto-boxing
15            cl1.addElement(Integer.valueOf(30));
16            cl1.removeElement(12);
17            System.out.println(cl1);
18
19    }
```

Generic method:

A **generic method** is a method that can work with **different data types** using a **type parameter**.

Example:

```
static public <T> T getData(T obj) {  
    return obj;  
}
```

Explanation:

- **<T>** → Type parameter.
It tells Java: “This method uses a type T.”
- **T getData(...)** → Return type is T.
The method returns the same type that it receives.
- **getData(T obj)** → Takes an argument of type T.
- **return obj** → Returns the same value.

In Java, we can restrict a generic type to only accept certain types using an upper bound.

Example:

```
3 import java.util.*;  
4  
5 public class CustomList<T extends List> {  
6     List<T> al=new ArrayList<T>();  
7  
8     public void addElement(T s) {  
9         al.add(s);  
10    }  
11  
12    public void removeElement(T s) {  
13        al.remove(s);  
14    }  
15  
16    public T get(int index) {  
17        return al.get(index);  
18    }
```

In this, T extends List, means T must be a List type.

So T can only be:

- ArrayList
- LinkedList
- Vector

- Any class that implements **java.util.List**

```
public class Main {  
    public static void main(String[] args) {  
  
        CustomList<ArrayList<Integer>> cl = new CustomList<>();  
  
        ArrayList<Integer> l1 = new ArrayList<>();  
        l1.add(10);  
        l1.add(20);  
  
        ArrayList<Integer> l2 = new ArrayList<>();  
        l2.add(100);  
        l2.add(200);  
  
        cl.add(l1);  
        cl.add(l2);  
  
        System.out.println(cl); // [[10, 20], [100, 200]]  
        System.out.println(cl.get(0)); // [10, 20]  
    }  
}
```